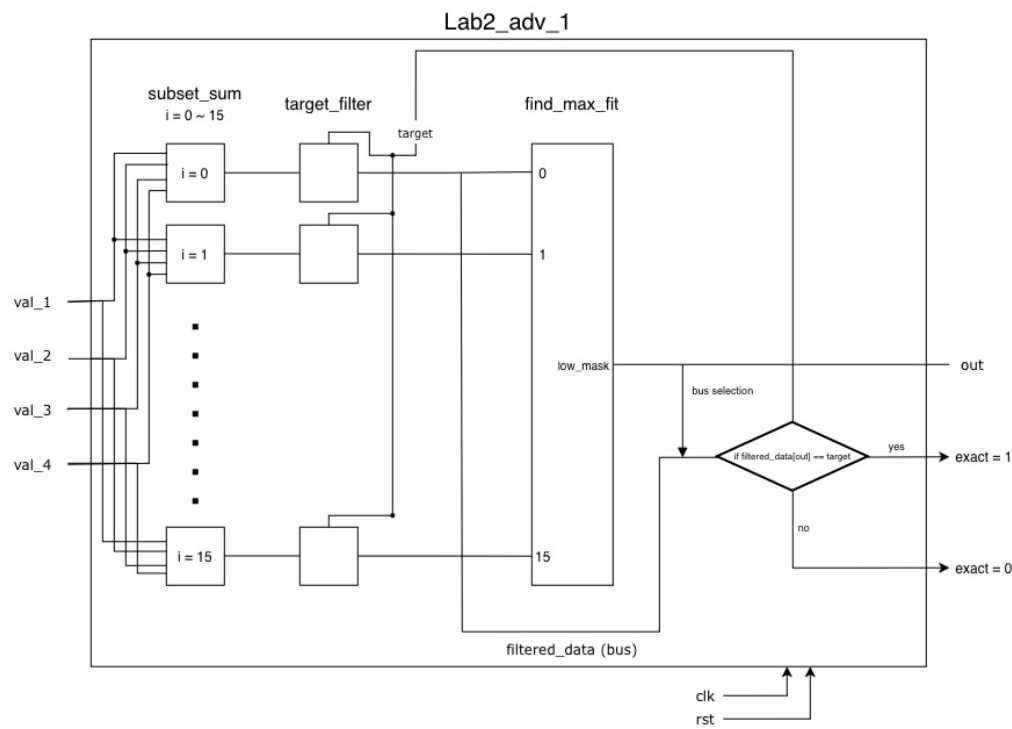


Advanced Lab Report – Lab 2

Student ID: 114062113	Name: 王宥傑
-----------------------	-----------

A. Architecture / Block Diagrams (30%)



B. Design & Implementation (20%)

Module name	Description
subset_sum	Calculate the sum of the given mask
target_filter	Filter sums greater than target
check_max_among	Check if the given sum is the greatest among others
find_max_fit	Instantiate 16 check_max_among modules to find the best-fit mask
lab2_adv_1	Main module

- subset_sum

Use 4 MUX to calculate the sum of given mask.

```
module subset_sum (
    input  logic [3:0] val_1, // 0
    input  logic [3:0] val_2,
    input  logic [3:0] val_3,
    input  logic [3:0] val_4, // 3
    input  logic [3:0] subset_number,
    output logic [5:0] sub_sum
);

    logic [3:0] selected_val1, selected_val2, selected_val3, selected_val4;

    mux m1(.a(4'b0), .b(val_1), .sel(subset_number[0]), .out(selected_val1));
    mux m2(.a(4'b0), .b(val_2), .sel(subset_number[1]), .out(selected_val2));
    mux m3(.a(4'b0), .b(val_3), .sel(subset_number[2]), .out(selected_val3));
    mux m4(.a(4'b0), .b(val_4), .sel(subset_number[3]), .out(selected_val4));

    assign sub_sum = selected_val1 + selected_val2 + selected_val3 + selected_val4;
endmodule
```

- target_filter

Filter sums greater than target.

```
module target_filter(
    input  logic [5:0] target,
    input  logic [5:0] sum,
    output logic [5:0] out
);

    assign out = (sum <= target) ? sum : 0;
endmodule
```

- check_max_among

Filter sums greater than target.

```
module check_max_among(
    input  [5:0] filtered_data [15:0],
    input  [3:0] index,
    output is_max
);

    logic [15:0] is_max_array;
    genvar i;
    for ( i = 0 ; i < 16 ; i = i + 1 ) begin
        assign is_max_array[i] = (filtered_data[index] >= filtered_data[i]) ? 1 : 0;
    end

    assign is_max = &is_max_array; // and all bits -> 1 bit
endmodule
```

- find_max_fit

Instantiate 16 check_max_among modules to find the best-fit mask

```
module find_max_fit(
    input  logic [5:0] filtered_data [15:0],
    output logic [3:0] out // best-fit mask
);

    logic [15:0] is_max;

    genvar i;
    for ( i = 0 ; i < 16 ; i = i + 1 )
        check_max_among my_check(filtered_data, i, is_max[i]);

    always @(*) begin
        if ( is_max[0] ) begin
            out = 0;
        end else if ( is_max[1] ) begin
            out = 1;
        end else if ( is_max[2] ) begin
            out = 2;
        end else if ( is_max[3] ) begin
            out = 3;
        end else if ( is_max[4] ) begin
            out = 4;
        end else if ( is_max[5] ) begin
            out = 5;
        end else if ( is_max[6] ) begin
            out = 6;
        end else if ( is_max[7] ) begin
            out = 7;
        end else if ( is_max[8] ) begin
            out = 8;
        end else if ( is_max[9] ) begin
            out = 9;
        end else if ( is_max[10] ) begin
            out = 10;
        end else if ( is_max[11] ) begin
            out = 11;
        end else if ( is_max[12] ) begin
            out = 12;
        end else if ( is_max[13] ) begin
            out = 13;
        end else if ( is_max[14] ) begin
            out = 14;
        end else if ( is_max[15] ) begin
            out = 15;
        end else
            out = 0;
    end

endmodule
```

• lab2_adv_1

Main solver

```
module lab2_adv_1 (
    input logic      clk,
    input logic      rst,
    input logic [3:0] val_1,
    input logic [3:0] val_2,
    input logic [3:0] val_3,
    input logic [3:0] val_4,
    input logic [5:0] target,
    output logic      exact,
    output logic [3:0] out
);

    logic [5:0] sub_sum_arr[15:0], filtered_data [15:0];

    genvar i;
    for ( i = 0 ; i < 16 ; i = i + 1 ) begin
        subset_sum my_subset_sum(
            val_1, val_2, val_3, val_4, 4'(i), sub_sum_arr[i]
        );
    end
    integer k;

    genvar j;
    for ( j = 0 ; j < 16 ; j = j + 1 ) begin
        target_filter my_tf(
            target, sub_sum_arr[j], filtered_data[j]
        );
    end

    logic [3:0] low_mask;
    find_max_fit my_find(filtered_data, low_mask);

    always_ff @( posedge clk ) begin
        if ( rst ) begin
            out <= 0;
            exact <= 0;
        end
        else begin
            out <= low_mask;
            if ( filtered_data[low_mask] == target )
                exact <= 1;
            else
                exact <= 0;
        end
    end

endmodule
```

C. Discussion & Analysis (40%)

- Specified Questions

1. Blocking assignments (“=”) are executed immediately, while non-blocking assignments (“<=”) work concurrently with other ones. Therefore, given that in a sequential circuit commands are synced with clock, blocking assignments should be used in combinational circuits, whereas non-blocking assignments should be used in sequential circuits.
2. If the problem grows to 16 values, we will have to check 2^{16} masks, which is 4096 times larger than the original problem. To design a sequential alternative, we only need a sum calculator and a counter that calculates and enumerates all the masks within 2^{16} cycles (one mask per cycle). As a result, compared with the combinational design where we need to instantiate 2^{16} modules, the sequential design is slower but lighter in terms of hardware.
3. Without the **valid** signal and the input registers, the circuit might load unexpected input from switches. For example, from inputtype “00” to “01,” as soon as we turn on the SW12, the combinational circuit will instantly load the current input from SW7-0, while we haven’t configured our expected numbers, contributing to bewildering computation.

- Problems Encountered and Solutions

In Problem 1, my design required a lot of module instantiations. To avoid time-consuming copy-and-paste steps, I looked up the tutorial of the usage of for-loop in SystemVerilog, and found it extremely useful.

- Design trade-offs

Since my design uses for-loop, the debugging process is very grueling, and the hardware implementation is more complicated than my friend’s.

- Possible improvements

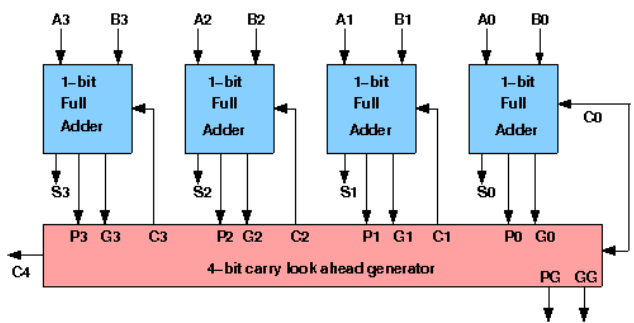
To make my design user-friendly, I would like to use the 7-segment display to show the user the current values or target stored in the registers when **valid** is high. In addition, I can also ask AI to improve my RTL design.

- AI usage

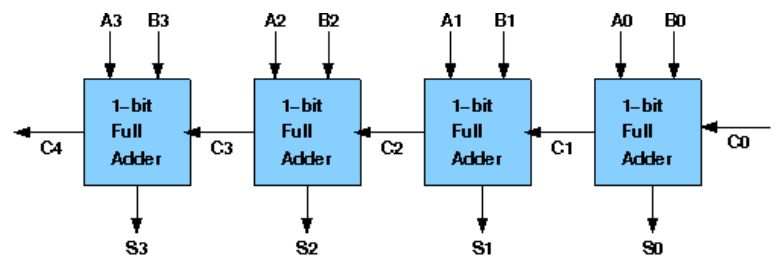
During the designing process, I used AI to help me debug some tiny errors, like treating output port as `input`. Even though AI can find those bugs in seconds, its response is sometimes wrong (e.g. giving wrong test cases). That’s why we always need to double-check and fact-check AI’s response.

D. Additional Work (Extra Credit: up to 10%)

Analyzing on my friend's code and synthesis, I found that if my friend's combination design leverages sequential chains (like multiple if-else with dependent variables), in the synthesis process, Vivado will warn developers that the combinational wiring might be slow due to numerous delays. On the other hand, since I used for-loop to instantiate tens of independent modules, the delays have little impact on the implementation of FPGA. Below is an analogy of the comparison between my design (CLA) and my friend's (RCA).



Carry Look-Ahead Adder ([src](#))



Ripple Carry Adder ([src](#))